

Lab: SVFIR and Control-Flow Reachability

(Week 2)

Yulei Sui

School of Computer Science and Engineering

University of New South Wales, Australia

Quiz-1 + Lab-Exercise-1 + Assignment-1

- A set of quizzes on WebCMS (5 points) due on **Week 3 Wednesday 23:59**
 - LLVM compiler and its intermediate representation
 - Code graphs (including ICFG and PAG)
- Lab-Exercise-1 (5 points) due on **Week 3 Wednesday 23:59**
 - Practice graph algorithms on **self-constructed graphs**
 - Implement graph reachability analysis on a general graph, and Andersen's points-to analysis using `solveWorklist` on a constraint graph.
- Assignment-1 (20 points) due on **Week 4 Wednesday 23:59**
 - **Control-flow**: Implement a context-sensitive graph traversal on a **CodeGraph** (i.e., ICFG) and collect **feasible** paths from a source to a sink node on the ICFG.
 - **Data-flow**: Implement field-sensitive Andersen's inclusion-based constraint solving for points-to analysis
 - Implement a taint checker **using control-flow and data-flow analysis**.

Quiz-1 + Lab-Exercise-1 + Assignment-1

- A set of quizzes on WebCMS (5 points) due on **Week 3 Wednesday 23:59**
 - LLVM compiler and its intermediate representation
 - Code graphs (including ICFG and PAG)
- Lab-Exercise-1 (5 points) due on **Week 3 Wednesday 23:59**
 - Practice graph algorithms on **self-constructed graphs**
 - Implement graph reachability analysis on a general graph, and Andersen's points-to analysis using `solveWorklist` on a constraint graph.
- Assignment-1 (20 points) due on **Week 4 Wednesday 23:59**
 - **Control-flow**: Implement a context-sensitive graph traversal on a **CodeGraph** (i.e., ICFG) and collect **feasible** paths from a source to a sink node on the ICFG.
 - **Data-flow**: Implement field-sensitive Andersen's inclusion-based constraint solving for points-to analysis
 - Implement a taint checker **using control-flow and data-flow analysis**.
 - **Specification and code template**: <https://github.com/SVF-tools/Software-Security-Analysis/wiki/Assignment-1>

Understanding LLVM-IR and SVF-IR

- (1) Compile C programs under SVFIR/src into their LLVM IR and print their SVF IR (PAG, ICFG, Constraint Graph).
 - <https://github.com/SVF-tools/Software-Security-Analysis/wiki/SVFIR>
 - Understand the mapping from a C program to its corresponding LLVM IR
 - Understand the mapping from LLVM IR to its corresponding SVF IR
- (2) Generate and visualize the graph representation of SVF IR (e.g., `example.ll.pag.dot`, `example.ll.icfg.dot`, `consG.ll.dot`).
 - <https://github.com/SVF-tools/Software-Security-Analysis/wiki/SVFIR#4-visualize-icfg-constraint-graph-and-svfirpag-graph>
- (3) Write code to iterate SVFVars and print nodes/edges of PAG and ICFG.
 - <https://github.com/SVF-tools/Software-Security-Analysis/blob/main/SVFIR/SVFIR.cpp#L74-L98> (C++)
 - <https://github.com/SVF-tools/SVF-Python/tree/main/demo> (Python)
- (4) More about LLVM IR and SVF's graph representation
 - LLVM language manual <https://llvm.org/docs/LangRef.html>
 - SVF website <https://github.com/SVF-tools/SVF>

Control-Flow Reachability

(Week 2)

Yulei Sui, Mohamad Barbar, Hanyuan Li, Angela He

School of Computer Science and Engineering

University of New South Wales, Australia

Algorithm overview

- Problem: context-insensitive reachability generates *impossible* paths

Algorithm overview

- Problem: context-insensitive reachability generates *impossible* paths
- Observation 1: in ICFGs, each call has corresponding return

Algorithm overview

- Problem: context-insensitive reachability generates *impossible* paths
- Observation 1: in ICFGs, each call has corresponding return
- Observation 2: inner functions "exit" before outer functions

Algorithm overview

- Problem: context-insensitive reachability generates *impossible* paths
- Observation 1: in ICFGs, each call has corresponding return
- Observation 2: inner functions "exit" before outer functions
- Solution: introduce **callstack** to track function calls/returns

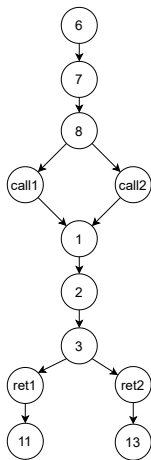
Context-Sensitive Control-Flow Reachability (Algorithm)

Algorithm 1: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode   snk : ICFGNode   path : vector(ICFGNode)
        callstack : vector(SVFInstruction)   visited : set(ICFGNode, callstack);

1  dfs(curNode, snk)
2  pair = <curNode, callstack>;
3  if pair  $\in$  visited then
4  |   return;
5  visited.insert(pair);
6  path.push_back(curNode);
7  if src == snk then
8  |   collectICFGPath(path);
9  foreach edge  $\in$  curNode.getOutEdges() do
10 | if edge.isIntraCFGEde() then
11 |   dfs(edge.dst, snk);
12 | else if edge.isCallCFGEde() then
13 |   callstack.push_back(edge.getCallSite());
14 |   dfs(edge.dst, snk);
15 |   callstack.pop_back();
16 | else if edge.isRetCFGEde() then
17 |   if callstack  $\neq \emptyset$  && callstack.back() == edge.getCallSite() then
18 |   |   callstack.pop_back();
19 |   |   dfs(edge.dst, snk);
20 |   |   callstack.push_back(edge.getCallSite());
21 |   else if callstack ==  $\emptyset$  then
22 |   |   dfs(edge.dst, snk);
23 visited.erase(pair);
24 path.pop_back();
```

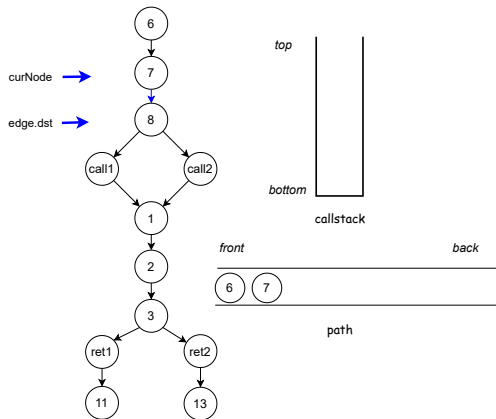
Example ICFG



The course repo can generate ICFG diagrams from LLVM IR files. To do so, follow the instructions at <https://github.com/SVF-tools/Software-Security-Analysis/wiki/SVFIR>.

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

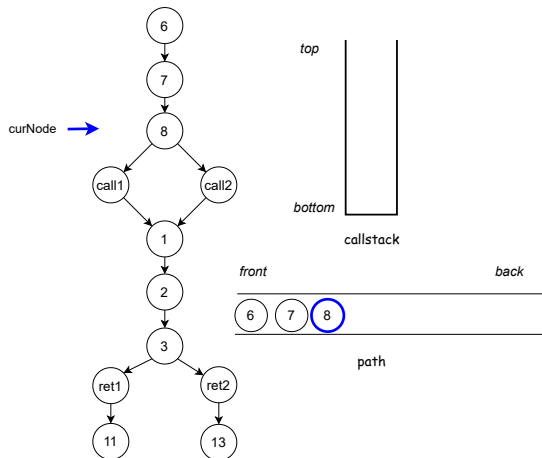


Algorithm 2: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = (curNode, callstack);  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23  visited.erase(pair);  
24  path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

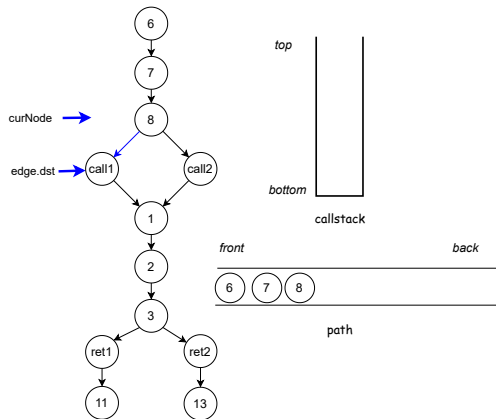


Algorithm 3: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;
1  dfs(curNode, snk)
2  pair = <curNode, callstack>;
3  if pair ∈ visited then
4  | return;
5  visited.insert(pair);
6  path.push_back(curNode);
7  if src == snk then
8  | collectICFGPath(path);
9  foreach edge ∈ curNode.getOutEdges() do
10 | if edge.isIntraCFGEde() then
11 | | dfs(edge.dst, snk);
12 | else if edge.isCallCFGEde() then
13 | | callstack.push_back(edge.getCallSite());
14 | | dfs(edge.dst, snk);
15 | | callstack.pop_back();
16 | else if edge.isRetCFGEde() then
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18 | | | callstack.pop_back();
19 | | | dfs(edge.dst, snk);
20 | | | callstack.push_back(edge.getCallSite());
21 | | else if callstack == ∅ then
22 | | | dfs(edge.dst, snk);
23 visited.erase(pair);
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

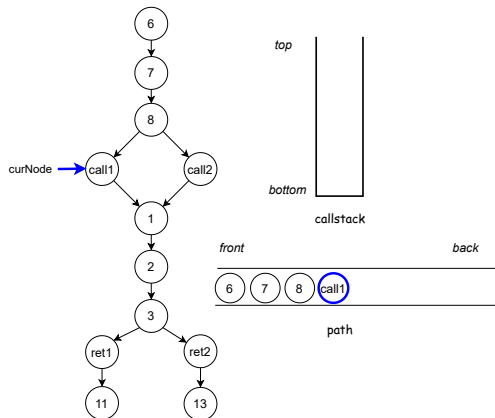


Algorithm 4: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = (curNode, callstack);  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23  visited.erase(pair);  
24  path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

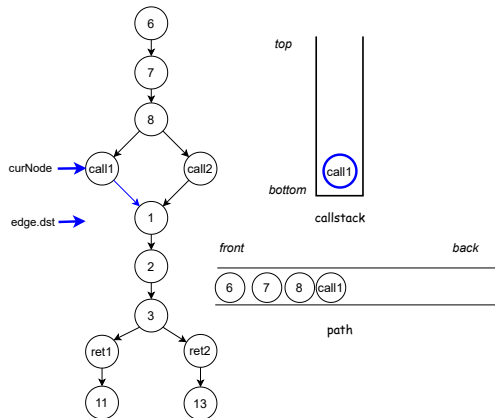


Algorithm 5: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;
1  dfs(curNode, snk)
2  pair = <curNode, callstack>;
3  if pair ∈ visited then
4  | return;
5  visited.insert(pair);
6  path.push_back(curNode);
7  if src == snk then
8  | collectICFGPath(path);
9  foreach edge ∈ curNode.getOutEdges() do
10 | if edge.isIntraCFGEde() then
11 | | dfs(edge.dst, snk);
12 | else if edge.isCallCFGEde() then
13 | | callstack.push_back(edge.getCallSite());
14 | | dfs(edge.dst, snk);
15 | | callstack.pop_back();
16 | else if edge.isRetCFGEde() then
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18 | | | callstack.pop_back();
19 | | | dfs(edge.dst, snk);
20 | | | callstack.push_back(edge.getCallSite());
21 | | else if callstack == ∅ then
22 | | | dfs(edge.dst, snk);
23 visited.erase(pair);
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



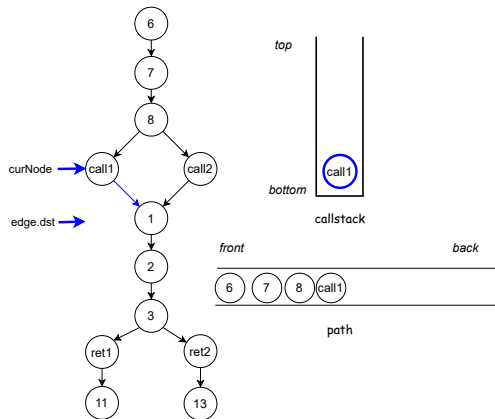
Algorithm 6: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;

1  dfs(curNode, snk)
2  pair = (curNode, callstack);
3  if pair ∈ visited then
4  | return;
5  visited.insert(pair);
6  path.push_back(curNode);
7  if src == snk then
8  | collectICFGPath(path);
9  foreach edge ∈ curNode.getOutEdges() do
10 | if edge.isIntraCFGEde() then
11 | | dfs(edge.dst, snk);
12 | else if edge.isCallCFGEde() then
13 | | callstack.push_back(edge.getCallSite());
14 | | dfs(edge.dst, snk);
15 | | callstack.pop_back();
16 | else if edge.isRetCFGEde() then
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18 | | | callstack.pop_back();
19 | | | dfs(edge.dst, snk);
20 | | | callstack.push_back(edge.getCallSite());
21 | | else if callstack == ∅ then
22 | | | dfs(edge.dst, snk);
23 visited.erase(pair);
24 path.pop_back();
```


Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

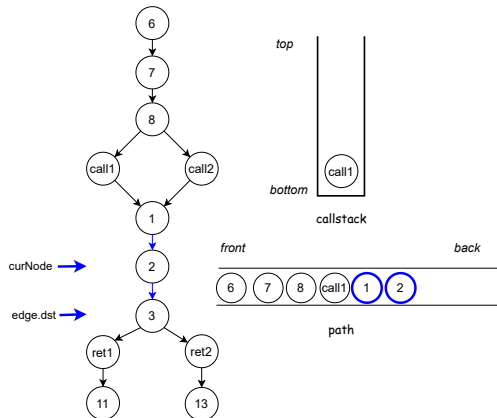


Algorithm 7: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = (curNode, callstack);  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23  visited.erase(pair);  
24  path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

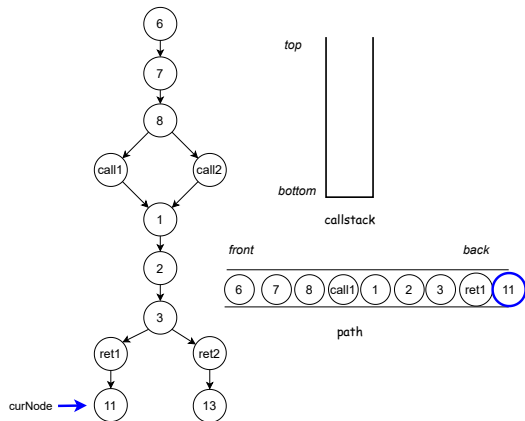


Algorithm 8: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = <curNode, callstack>;  
3  if pair ∈ visited then  
4  |   return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  |   collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEdge() then  
11 | |   dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEdge() then  
13 | |   callstack.push_back(edge.getCallSite());  
14 | |   dfs(edge.dst, snk);  
15 | |   callstack.pop_back();  
16 | else if edge.isRetCFGEdge() then  
17 | |   if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | |   callstack.pop_back();  
19 | | |   dfs(edge.dst, snk);  
20 | | |   callstack.push_back(edge.getCallSite());  
21 | |   else if callstack == ∅ then  
22 | | |   dfs(edge.dst, snk);  
23 visited.erase(pair);  
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

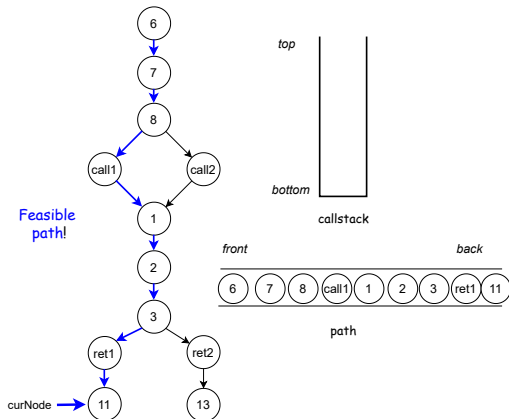


Algorithm 9: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;
1  dfs(curNode, snk)
2  pair = <curNode, callstack>;
3  if pair ∈ visited then
4  | return;
5  visited.insert(pair);
6  path.push_back(curNode);
7  if src == snk then
8  | collectICFGPath(path);
9  foreach edge ∈ curNode.getOutEdges() do
10 | if edge.isIntraCFGEde() then
11 | | dfs(edge.dst, snk);
12 | else if edge.isCallCFGEde() then
13 | | callstack.push_back(edge.getCallSite());
14 | | dfs(edge.dst, snk);
15 | | callstack.pop_back();
16 | else if edge.isRetCFGEde() then
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18 | | | callstack.pop_back();
19 | | | dfs(edge.dst, snk);
20 | | | callstack.push_back(edge.getCallSite());
21 | | else if callstack == ∅ then
22 | | | dfs(edge.dst, snk);
23 visited.erase(pair);
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



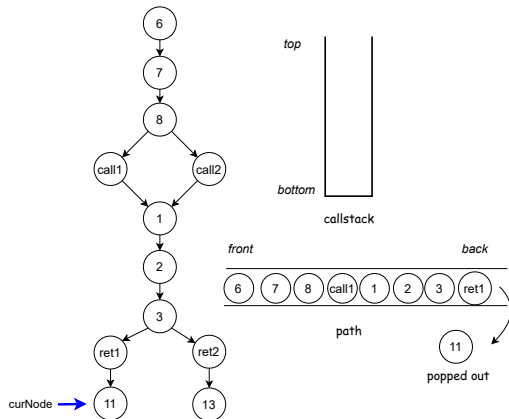
Algorithm 10: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;

1  dfs(curNode, snk)
2  pair = (curNode, callstack);
3  if pair ∈ visited then
4  | return;
5  visited.insert(pair);
6  path.push_back(curNode);
7  if src == snk then
8  | collectICFGPath(path);
9  foreach edge ∈ curNode.getOutEdges() do
10 | if edge.isIntraCFGEde() then
11 | | dfs(edge.dst, snk);
12 | else if edge.isCallCFGEde() then
13 | | callstack.push_back(edge.getCallSite());
14 | | dfs(edge.dst, snk);
15 | | callstack.pop_back();
16 | else if edge.isRetCFGEde() then
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18 | | | callstack.pop_back();
19 | | | dfs(edge.dst, snk);
20 | | | callstack.push_back(edge.getCallSite());
21 | | else if callstack == ∅ then
22 | | | dfs(edge.dst, snk);
23 visited.erase(pair);
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

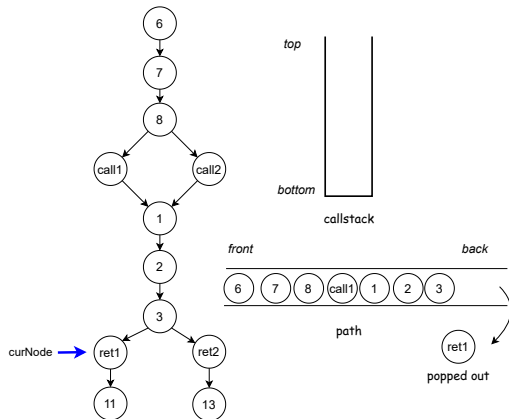


Algorithm 11: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = <curNode, callstack>;  
3  if pair ∈ visited then  
4  |   return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  |   collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 |   if edge.isIntraCFGEde() then  
11 |   |   dfs(edge.dst, snk);  
12 |   else if edge.isCallCFGEde() then  
13 |   |   callstack.push_back(edge.getCallSite());  
14 |   |   dfs(edge.dst, snk);  
15 |   |   callstack.pop_back();  
16 |   else if edge.isRetCFGEde() then  
17 |   |   if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 |   |   |   callstack.pop_back();  
19 |   |   |   dfs(edge.dst, snk);  
20 |   |   |   callstack.push_back(edge.getCallSite());  
21 |   |   else if callstack == ∅ then  
22 |   |   |   dfs(edge.dst, snk);  
23  visited.erase(pair);  
24  path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

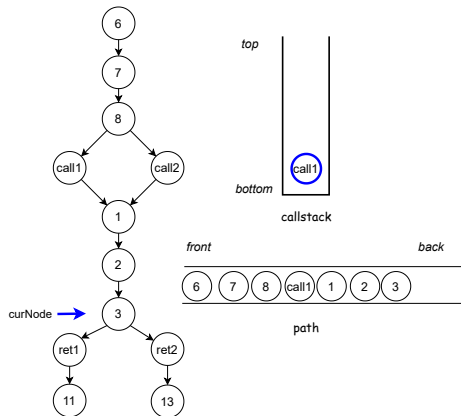


Algorithm 12: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = (curNode, callstack);  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23 | visited.erase(pair);  
24 | path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



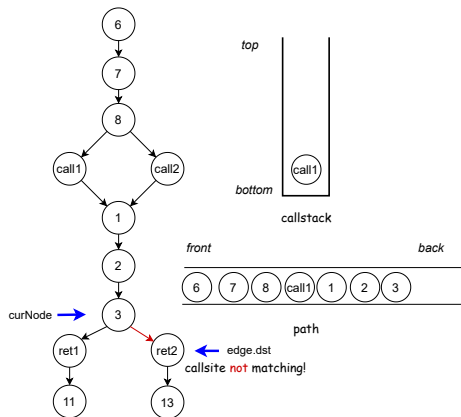
Algorithm 13: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;

1  dfs(curNode, snk)
2  pair = <curNode, callstack>;
3  if pair ∈ visited then
4  | return;
5  visited.insert(pair);
6  path.push_back(curNode);
7  if src == snk then
8  | collectICFGPath(path);
9  foreach edge ∈ curNode.getOutEdges() do
10 | if edge.isIntraCFGEde() then
11 | | dfs(edge.dst, snk);
12 | else if edge.isCallCFGEde() then
13 | | callstack.push_back(edge.getCallSite());
14 | | dfs(edge.dst, snk);
15 | | callstack.pop_back();
16 | else if edge.isRetCFGEde() then
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18 | | | callstack.pop_back();
19 | | | dfs(edge.dst, snk);
20 | | | callstack.push_back(edge.getCallSite());
21 | | else if callstack == ∅ then
22 | | | dfs(edge.dst, snk);
23 visited.erase(pair);
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



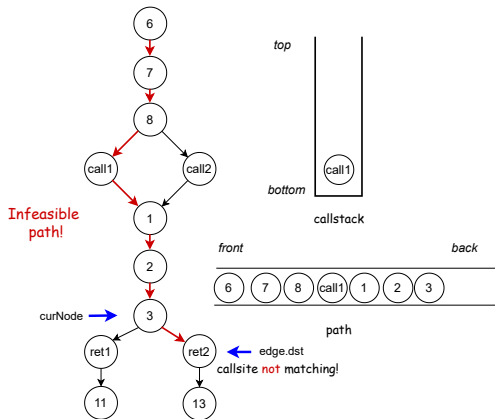
Algorithm 14: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;

1 dfs(curNode, snk)
2   pair = <curNode, callstack>;
3   if pair ∈ visited then
4     return;
5   visited.insert(pair);
6   path.push_back(curNode);
7   if src == snk then
8     collectICFGPath(path);
9   foreach edge ∈ curNode.getOutEdges() do
10    if edge.isIntraCFGEde() then
11      dfs(edge.dst, snk);
12    else if edge.isCallCFGEde() then
13      callstack.push_back(edge.getCallSite());
14      dfs(edge.dst, snk);
15      callstack.pop_back();
16    else if edge.isRetCFGEde() then
17      if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18        callstack.pop_back();
19        dfs(edge.dst, snk);
20        callstack.push_back(edge.getCallSite());
21      else if callstack == ∅ then
22        dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```


Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



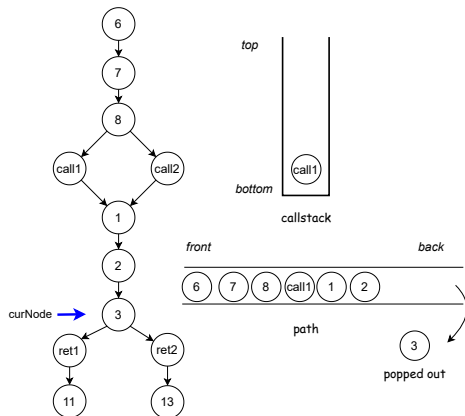
Algorithm 15: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;

1  dfs(curNode, snk)
2  pair = <curNode, callstack>;
3  if pair ∈ visited then
4  | return;
5  visited.insert(pair);
6  path.push_back(curNode);
7  if src == snk then
8  | collectICFGPath(path);
9  foreach edge ∈ curNode.getOutEdges() do
10 | if edge.isIntraCFGEde() then
11 | | dfs(edge.dst, snk);
12 | else if edge.isCallCFGEde() then
13 | | callstack.push_back(edge.getCallSite());
14 | | dfs(edge.dst, snk);
15 | | callstack.pop_back();
16 | else if edge.isRetCFGEde() then
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18 | | | callstack.pop_back();
19 | | | dfs(edge.dst, snk);
20 | | | callstack.push_back(edge.getCallSite());
21 | | else if callstack == ∅ then
22 | | | dfs(edge.dst, snk);
23 visited.erase(pair);
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

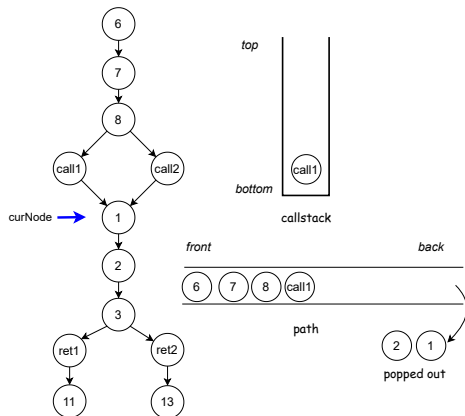


Algorithm 16: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = (curNode, callstack);  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23 | visited.erase(pair);  
24 | path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

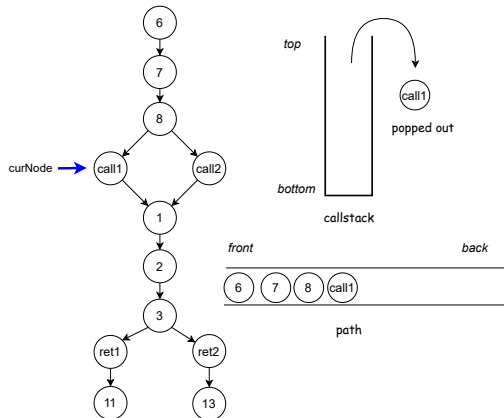


Algorithm 17: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = (curNode, callstack);  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23  visited.erase(pair);  
24  path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

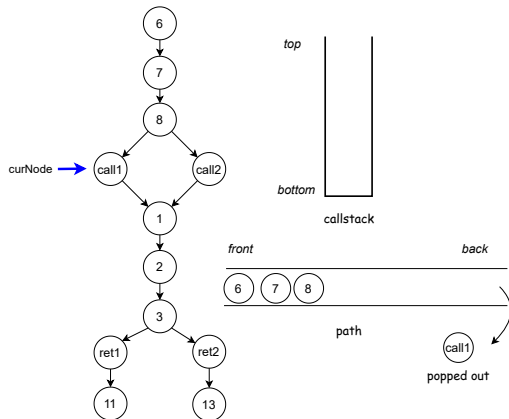


Algorithm 18: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = <curNode, callstack>;  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23  visited.erase(pair);  
24  path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

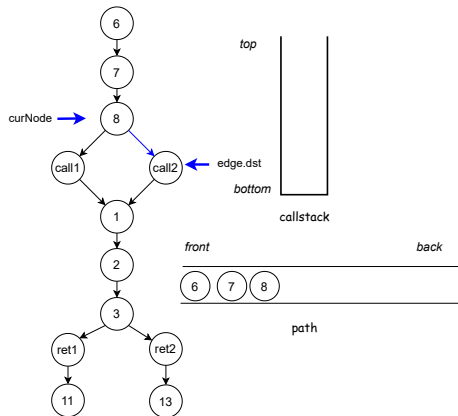


Algorithm 19: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = (curNode, callstack);  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23 visited.erase(pair);  
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

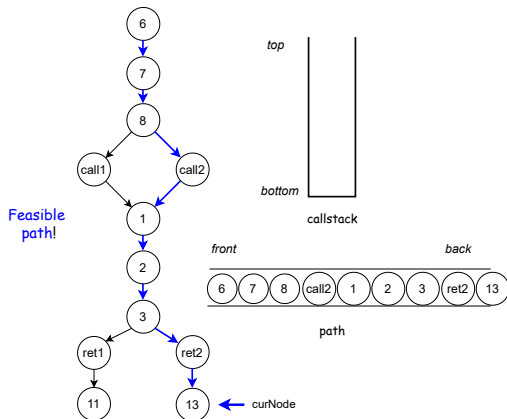


Algorithm 20: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = <curNode, callstack>;  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23  visited.erase(pair);  
24  path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

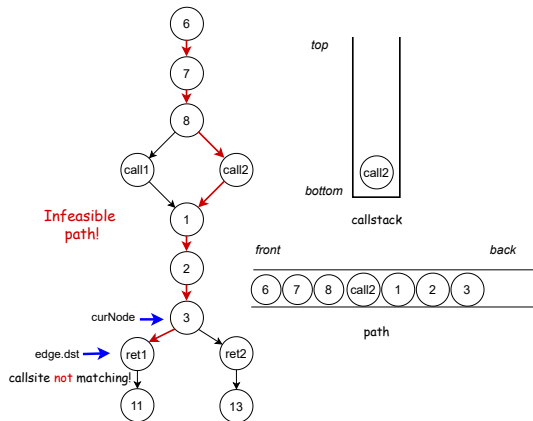


Algorithm 21: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode   snk : ICFGNode   path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>   visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = <curNode, callstack>;  
3  if pair ∈ visited then  
4  |   return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  |   collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | |   dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | |   callstack.push_back(edge.getCallSite());  
14 | |   dfs(edge.dst, snk);  
15 | |   callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | |   if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | |   callstack.pop_back();  
19 | | |   dfs(edge.dst, snk);  
20 | | |   callstack.push_back(edge.getCallSite());  
21 | |   else if callstack == ∅ then  
22 | | |   dfs(edge.dst, snk);  
23 visited.erase(pair);  
24 path.pop_back();
```

Context-Sensitive Control-Flow Reachability

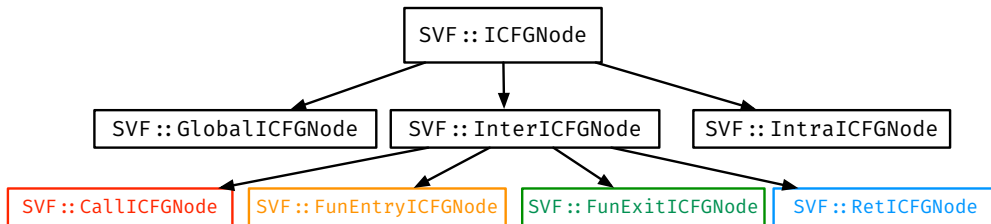
A feasible path from node 6 to node 11 on ICFG



Algorithm 22: 1 Context sensitive control-flow reachability

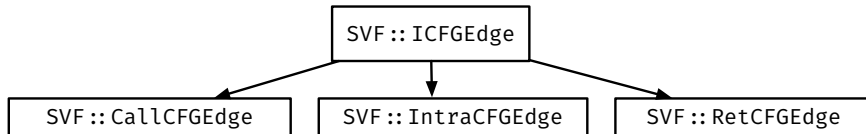
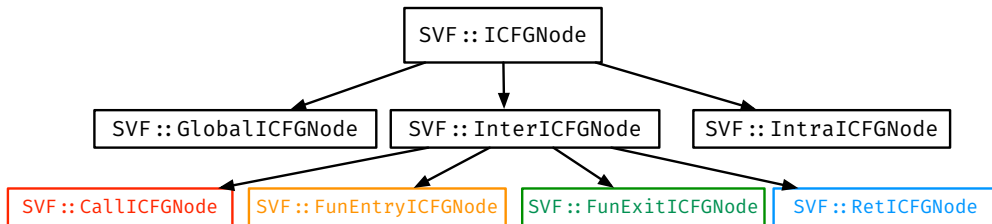
```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector<ICFGNode>  
        callstack : vector<SVFInstruction>  visited : set<ICFGNode, callstack>;  
1  dfs(curNode, snk)  
2  pair = {curNode, callstack};  
3  if pair ∈ visited then  
4  | return;  
5  visited.insert(pair);  
6  path.push_back(curNode);  
7  if src == snk then  
8  | collectICFGPath(path);  
9  foreach edge ∈ curNode.getOutEdges() do  
10 | if edge.isIntraCFGEde() then  
11 | | dfs(edge.dst, snk);  
12 | else if edge.isCallCFGEde() then  
13 | | callstack.push_back(edge.getCallSite());  
14 | | dfs(edge.dst, snk);  
15 | | callstack.pop_back();  
16 | else if edge.isRetCFGEde() then  
17 | | if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then  
18 | | | callstack.pop_back();  
19 | | | dfs(edge.dst, snk);  
20 | | | callstack.push_back(edge.getCallSite());  
21 | | else if callstack == ∅ then  
22 | | | dfs(edge.dst, snk);  
23 visited.erase(pair);  
24 path.pop_back();
```


ICFG Node and Edge Classes



<https://github.com/SVF-tools/SVF/blob/master/include/Graphs/ICFGNode.h>

ICFG Node and Edge Classes



<https://github.com/SVF-tools/SVF/blob/master/include/Graphs/ICFGE.h>

SVFUtil::cast and SVFUtil::dyn_cast

- C++ Inheritance: see slides in Week 1 Lab.
- Casting a **parent** class pointer to pointer of a **Child** type:
 - `SVFUtil::cast`
 - Casts a pointer or reference to an instance of a specified class. This cast fails and aborts the program if the object or reference is not the specified class at runtime.
 - `SVFUtil::dyn_cast`
 - "Checked cast" operation. Checks to see if the operand is of the specified type, and if so, returns a pointer to it (this operator does not work with references). If the operand is not of the correct type, a null pointer is returned.
 - Works very much like the `dynamic_cast<>` operator in C++, and should be used in the same circumstances.
- Example: accessing the attributes of the child class via casting.
 - `RetICFGNode* retNode = SVFUtil::cast<RetICFGNode>(ICFGNode);`
 - `CallCFGEde* callEdge = SVFUtil::dyn_cast<CallCFGEde>(ICFGEde);`

“Casting” from Parent to Child in Python

- Python is a **dynamically typed language**:
 - No cast is required to call a method or access an attribute of an object.
 - A method can be called without declaring the object's type in code, as long as the object is of the correct type at runtime.
- **Dynamic Typing $\neq$ No Discipline**:
 - **Adding type hints** is highly recommended to improve **readability** and **tooling support**.
 - This enables code completion, bug detection, and better maintainability.
- `isinstance()` and `typing.cast`:
 - Use `isinstance()` to narrow the type at runtime and assist IDE.
 - Use `typing.cast()` to explicitly annotate the expected type.
- Example: narrowing and annotating types for better IDE inference.
 - `if isinstance(node, RetICFGNode):` # IDE infers `node` is `RetICFGNode`
 - `call_edge = typing.cast(CallCFGEdge, edge)` # IDE infers `call_edge` is `CallCFGEdge`